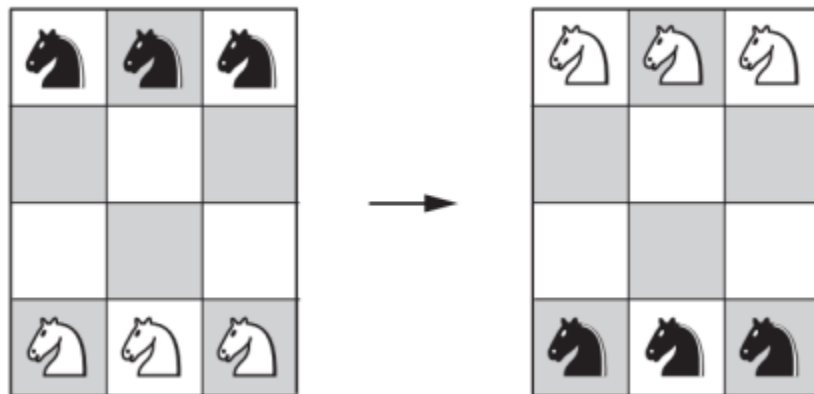


Task 4

Task Description:

In this task we have a 6 knights 3 white and 3 black knights that should interchange between each other so that the white should go to the top 3 columns and the black knights should go to the bottom three and during the movement of the knights no 2 knights should share the same block and here we are assigned to design an iterative improvement algorithm for this problem



Assumptions:

Board & Setup

- The board is always a fixed 4×3 board
- Initial state is always Black knights on top row, White knights on bottom row, middle rows empty
- Goal state is always Whites on top, Blacks on bottom

Piece Movement

- Knights move in standard L-shape (chess rules)
- A knight can only move to an empty cell (.)
- Knights cannot capture or jump over each other (only move to empty)
- B and W knights move in the same turn

Algorithm

- Used iterative improvement algorithm : Hill climbing and Simulated Annealing
- Hill Climbing may **not** find the optimal solution
- Simulated Annealing was implemented as an improved iterative improvement algorithm
- BFS always finds the **shortest path** (optimal solution)
- Random restarts are allowed in Hill Climbing to escape local minima
- The puzzle is always **solvable** from the initial state

Implementation

- The board is represented as a char [4][3] array
- States are encoded as 12-character strings for easy comparison and storage
- Helper functions is implemented to help convert it from string to board and vice versa
- 'B' = black knight, 'W' = white knight, '.' = empty cell

Pseudocode (for Hill Climbing):

1. getValidMoves(board, row, col)

SET validMoves = empty list

knight movements:

(row+2, col+1), (row+2, col-1)

(row-2, col+1), (row-2, col-1)

(row+1, col+2), (row+1, col-2)

(row-1, col+2), (row-1, col-2)

FOR each move (newRow, newCol):

IF newRow and newCol are within board bounds:

IF board[newRow][newCol] == '.':

ADD (newRow, newCol) to validMoves

RETURN validMoves

2. fromString(state, board)

SET counter = 0

FOR i from 0 to 3:

FOR j from 0 to 2:

board[i][j] = state[counter]

counter++

3. fromBoard(board)

SET state = empty string

FOR i from 0 to 3:

 FOR j from 0 to 2:

 APPEND board[i][j] to state

RETURN state

4. score(board, target)

SET score = 0, distance = 0

CONVERT target string to targetBoard

FOR each cell (i,j) in board:

 IF board[i][j] == 'W': distance += |i - 3|

 IF board[i][j] == 'B': distance += |i - 0|

 IF board[i][j] == targetBoard[i][j] AND not '.': score++

RETURN 18 - distance + score

5. nextState(board)

SET states = empty list

FOR each cell (i,j) in board:

 IF board[i][j] == 'B' OR 'W':

 GET validMoves = getValidMoves(board, i, j)

 FOR each move in validMoves:

 COPY board to newBoard

SWAP newBoard[i][j] with newBoard[move.row][move.col]

CONVERT newBoard to string

ADD to states

RETURN states

6. randomShuffle(state)

CONVERT state to board

GET all nextStates from board

RETURN a random state from nextStates

7. printBoard(board)

FOR i from 0 to 3:

 FOR j from 0 to 2:

 PRINT board[i][j]

 PRINT newline

8. hillClimb(board, target)

SET currentState = fromBoard(board)

SET path = [currentState]

SET sidewaysCount = 0, restarts = 0

SET MAX_SIDEWAYS = 200, MAX_RESTARTS = 200

WHILE true:

 IF currentState == target:

 PRINT "Solution found in path.size() steps"

FOR each state in path:

PRINT step number

PRINT board

RETURN

GET nextStates from currentState

SET bestState = currentState

SET bestScore = score(board, target)

FOR each state in nextStates:

IF score(state) > bestScore:

bestScore = score(state)

bestState = state

sidewaysCount = 0

IF bestState == currentState: // stuck

IF sidewaysCount < MAX_SIDEWAYS:

GET all equal scoring states

PICK random one as currentState

sidewaysCount++

ELSE: // restart

restarts++

IF restarts > MAX_RESTARTS:

PRINT "No solution found"

RETURN

FOR 40 times: currentState = randomShuffle(currentState)

CLEAR path

sidewaysCount = 0

ELSE:

currentState = bestState

ADD to path

Pseudocode (for BFS):

1. **getValidMoves(board, row, col)**

SET validMoves = empty list

knight movements :

(row+2, col+1), (row+2, col-1)

(row-2, col+1), (row-2, col-1)

(row+1, col+2), (row+1, col-2)

(row-1, col+2), (row-1, col-2)

FOR each move (newRow, newCol):

IF newRow and newCol are within board bounds:

IF board[newRow][newCol] == '.':

ADD (newRow, newCol) to validMoves

RETURN validMoves

2. **fromString(state, board)**

SET counter = 0

FOR i from 0 to 3:

FOR j from 0 to 2:

board[i][j] = state[counter]

counter++

3. fromBoard(board)

SET state = empty string

FOR i from 0 to 3:

 FOR j from 0 to 2:

 APPEND board[i][j] to state

RETURN state

4. nextState(board)

SET states = empty list

FOR each cell (i,j) in board:

 IF board[i][j] == 'B' OR 'W':

 GET validMoves = getValidMoves(board, i, j)

 FOR each move in validMoves:

 COPY board to newBoard

 SWAP newBoard[i][j] with newBoard[move.row][move.col]

 CONVERT newBoard to string

 ADD to states

RETURN states

5. breadthFirstSearch(board, target)

SET start = fromBoard(board)

SET visited = empty set

SET parent = empty map

SET queue = empty queue

ADD start to queue

ADD start to visited

WHILE queue is not empty:

 GET currentState from front of queue

 REMOVE from front of queue

 IF currentState == target:

 RETURN currentState

 CONVERT currentState to currentBoard

 GET nextStates = nextState(currentBoard)

 FOR each state in nextStates:

 IF state NOT in visited:

 ADD state to visited

 ADD state to queue

 SET parent[state] = currentState

RETURN "no solution found"

6. printBoard(board)

FOR i from 0 to 3:

 FOR j from 0 to 2:

 PRINT board[i][j]

 PRINT newline

Pseudocode (for Simulated Annealing):

1. **getValidMoves(board, row, col)**

SET validMoves = empty list

knight movements:

(row+2, col+1), (row+2, col-1)

(row-2, col+1), (row-2, col-1)

(row+1, col+2), (row+1, col-2)

(row-1, col+2), (row-1, col-2)

FOR each move (newRow, newCol):

IF newRow and newCol are within board bounds:

IF board[newRow][newCol] == '.':

ADD (newRow, newCol) to validMoves

RETURN validMoves

2. **fromString(state, board)**

SET counter = 0

FOR i from 0 to 3:

FOR j from 0 to 2:

board[i][j] = state[counter]

counter++

3. **fromBoard(board)**

SET state = empty string

```
FOR i from 0 to 3:  
  FOR j from 0 to 2:  
    APPEND board[i][j] to state  
RETURN state
```

4. score(board, target)

```
SET score = 0, distance = 0  
CONVERT target string to targetBoard
```

```
FOR each cell (i,j) in board:  
  IF board[i][j] == 'W': distance += |i - 3|  
  IF board[i][j] == 'B': distance += |i - 0|  
  IF board[i][j] == targetBoard[i][j] AND not '.': score++  
  
RETURN 18 - distance + score
```

5. nextState(board)

```
SET states = empty list  
  
FOR each cell (i,j) in board:  
  IF board[i][j] == 'B' OR 'W':  
    GET validMoves = getValidMoves(board, i, j)  
    FOR each move in validMoves:  
      COPY board to newBoard  
      SWAP newBoard[i][j] with newBoard[move.row][move.col]  
      CONVERT newBoard to string
```

ADD to states

RETURN states

6. randomShuffle(state)

CONVERT state to board

GET all nextStates from board

RETURN a random state from nextStates

7. printBoard(board)

FOR i from 0 to 3:

FOR j from 0 to 2:

PRINT board[i][j]

PRINT newline

8. SET currentState = fromBoard(board)

SET path = [currentState]

SET restarts = 0

SET MAX_RESTARTS = 200

SET temperature = 1000.0 // tolerance

SET coolingRate = 0.995 //tolerance decrease rate

SET minTemperature = 0.1 // minimum tolerance

WHILE true:

IF currentState == target:

PRINT "Solution found in path.size() steps"

FOR each state in path:

PRINT step number

PRINT board

RETURN

GET nextStates = nextState(board)

PICK a random state from nextStates

SET currentScore = score(board, target)

SET randomScore = score(randomState, target)

SET delta = randomScore - currentScore

IF delta > 0: // better state

accept it

currentState = randomState

ADD currentState to path

ELSE: // worse state

SET probability = $e^{(\text{delta}/\text{temperature})}$

SET randomNumber = random number between 0 and 1

IF randomNumber < probability: // accept with probability

currentState = randomState

ADD currentState to path

temperature = temperature × coolingRate // decrease the temperature

IF temperature < minTemperature:

restarts++

IF restarts > MAX_RESTARTS:

PRINT "No solution found"

RETURN

SET temperature = 1000.0

FOR 40 times:

currentState = randomShuffle(currentState)

CLEAR path

ADD currentState to path

Code implementation (helper functions used by all the codes):

1.imports and initialization

```
#include <iostream>
```

```
#include <vector>
```

```
#include <utility>
```

```
#include <queue>
```

```
#include <set>
```

```
#include <map>
```

```
#include <algorithm>
```

```
#include <cstdlib>
```

```
#include <random>
```

```
#include <ctime>
```

```
using namespace std;
```

```
void intitalize (){
```

```
    char arr[4][3]={
```

```
        {'B','B','B'}, // (top)
```

```
        {'.', '.', '.'},
```

```
        {'.', '.', '.'},
```

```
        {'W','W','W'} // (bottom)
```

```
}
```

```
;}



---


```

```

2.vector<pair<int,int>> getValidMoves(char board[4][3], int row, int col) {
    vector<pair<int,int>> validMoves;
    if(row+2 < 4 && col+1 < 3 && board [row+2][col+1] == '.'){
        validMoves.push_back(make_pair(row+2,col+1)); // up right
    }
    if(row+2 < 4 && col-1 >= 0 && board [row+2][col-1] == '.'){
        validMoves.push_back(make_pair(row+2,col-1)); // up left
    }
    if(row-2 >= 0 && col-1 >= 0 && board [row-2][col-1] == '.'){
        validMoves.push_back(make_pair(row-2,col-1)); // down left
    }
    if(row-2 >= 0 && col+1 < 3 && board [row-2][col+1] == '.'){
        validMoves.push_back(make_pair(row-2,col+1)); // down right
    }
    if(row+1 < 4 && col+2 < 3 && board [row+1][col+2] == '.'){
        validMoves.push_back(make_pair(row+1,col+2)); // right up
    }
    if(row-1 >= 0 && col+2 < 3 && board [row-1][col+2] == '.'){
        validMoves.push_back(make_pair(row-1,col+2)); // right down
    }
    if(row+1 < 4 && col-2 >= 0 && board [row+1][col-2] == '.'){
        validMoves.push_back(make_pair(row+1,col-2)); // left up
    }
    if(row-1 >= 0 && col-2 >= 0 && board [row-1][col-2] == '.'){
        validMoves.push_back(make_pair(row-1,col-2)); // left down
    }
}

```

```
return validMoves; }
```

```
3. void fromstring(string state, char arr[4][3] ) {  
    int counter = 0;  
    for (int i = 0; i < 4; i++) {  
        for (int j = 0; j < 3; j++) {  
            arr[i][j] = state[counter++];  
        }  
    }  
}
```

```
4. string fromboard(char board[4][3]){  
    string state;  
    for (int i = 0; i < 4; i++) {  
        for (int j = 0; j < 3; j++) {  
            state += board[i][j];  
        }  
    }  
    return state;  
}
```

5. int score(char board[4][3],string target){ //used by simulated annealing and hill climbing

int score =0;

int distance = 0;

char targetboard[4][3] ;

fromstring(target,targetboard);

for (int i=0;i<4;i++){

for(int j =0 ; j<3 ; j++){

if (board[i][j] == 'W') {

distance += abs(i - 3);

} else if (board[i][j] == 'B') {

distance += abs(i - 0);

}

if (board[i][j] == targetboard[i][j] && board[i][j] != '.') {

score++;

}

}

}

return 18-distance+score;

}

```

6. vector<string> nextstate( char arr[4][3] ){
    int i,j;
    vector<string> states;
    for (i=0;i < 4 ;i++){
        for (j=0;j <3; j++){
            if (arr[i][j] == 'B' || arr[i][j] == 'W'){
                vector<pair<int,int>> validMoves = getValidMoves(arr, i, j);
                for (const auto& move : validMoves) {
                    char newState[4][3];
                    memcpy(newState, arr, sizeof(newState));
                    swap(newState[i][j], newState[move.first][move.second]);
                    string newStateStr = fromboard(newState);
                    states.push_back(newStateStr);
                }
            }
        }
    }
    return states;
}

```

```

7. string randomshuffle(string state){ //used by simulated annealing and hill climbing
    vector <string> nextStates;
    char board[4][3];
    fromstring(state, board);
    nextStates = nextstate(board);
}

```

```
return nextStates[rand() % nextStates.size()];  
}
```

```
8. void printboard(char board[4][3]){  
    for (int i = 0; i < 4; i++) {  
        for (int j = 0; j < 3; j++) {  
            cout << board[i][j] << " ";  
        }  
  
        cout << endl;  
    }  
    return;  
}
```

Code implementation (Hill Climbing):

```
void hillclimb(char board[4][3], string target){  
    vector<string> path;  
    int sidewaysCount = 0;  
    int MAX_SIDEWAYS = 200;  
    int restarts = 0;  
    int i = 1;  
    int MAX_RESTARTS = 200;  
    string currentState = fromboard(board);  
    path.push_back(currentState);
```

```

while (true) {
    if (currentState == target) {
        cout << "Solution found in " << path.size() << " steps." << endl;

        int step = 1;

        for (const auto& state : path) {
            fromstring(state, board);

            cout << "--- step " << step++ << " ---" << endl;

            printboard(board);
        }

        return;
    }

    else {
        vector<string> nextStates = nextstate(board);

        string bestState = currentState;

        int bestScore = score(board,target);

        for (const auto& state : nextStates) {
            char newBoard[4][3];

            fromstring(state, newBoard);

            int newScore = score(newBoard,target);

            if (newScore > bestScore) {
                bestScore = newScore;

                bestState = state;

                sidewaysCount=0;
            }
        }
    }
}

```

```

        if (bestState == currentState) {
if (sidewaysCount < MAX_SIDEWAYS) {
    vector<string> equalStates;
    for (const auto& state : nextStates) {
        char newBoard[4][3];
        fromstring(state, newBoard);
        if (score(newBoard, target) == bestScore) {
            equalStates.push_back(state);
        }
    }
    if (!equalStates.empty()) {
        currentState = equalStates[rand() % equalStates.size()];
        fromstring(currentState, board);
        path.push_back(currentState);
        sidewaysCount++;
    }
    else {
        sidewaysCount = MAX_SIDEWAYS;
    }else {
        restarts++;
    }
if (restarts > MAX_RESTARTS) {
    cout << "Maximum restarts reached. NO solution found." << endl;
cout << "Total restarts attempted: " << restarts << endl;
    cout << "tried " << path.size() << " steps." << endl;
    return; }
//randomize board to try to escape local minumum

```



```

for (int k = 0; k < 40; k++) {
    currentState = randomshuffle(currentState);
}
fromstring(currentState, board);
path.clear();
path.push_back(currentState);
sidewaysCount = 0;
}
}

```

```

    else {
        currentState = bestState;
        path.push_back(currentState);
        fromstring(currentState, board);
    }
}
}}

```

```

int main() {
    srand(time(nullptr));
    char board[4][3] = {
        {'B','B','B'},
        {'.','.', '.'},
        {'.','.', '.'},
    }
}

```

```

        {'W','W','W'}
    };

    cout << "starting state:" << endl;

    printboard(board);

    string target = "WWW.....BBB";

    hillclimb(board, target);

    return 0;
}

```

Complexity

Time complexity: $O(R \times S)$

Space complexity : $O(1)$ // it doesn't save old states it only cares about the current one

R = max restarts

S = max sideways moves

Code implementation (BFS):

```

string breadthfirstsearch(char board[4][3], string target){

    set<string> visited;

    map<string,string> parent;

    queue<string> q;

    string start = fromboard(board);

    q.push(start);

    visited.insert(start);

    while (!q.empty()) {

```

```

string currentState = q.front();
q.pop();
if (currentState == target) {
    return currentState; // Found the target state
}
char currentBoard[4][3];
fromstring(currentState, currentBoard);
vector<string> nextStates = nextstate(currentBoard);
for (const string& nextState : nextStates) {
    if (visited.find(nextState) == visited.end()) {
        visited.insert(nextState);
        q.push(nextState);
        parent[nextState]= currentState; // Store the parent state for path reconstruction
    }
}
return "no solution found";
}

```

```

int main() {
    char board[4][3] = {
        {'B','B','B'},
        {'.','.', '.'},
        {'.','.', '.'},
        {'W','W','W'}
    };
    cout << "-----Breadth-First Search-----" << endl;

```

```
    cout << "starting state:" << endl;
    printboard(board);
    cout << "-----" << endl;
    string target = "WWW.....BBB";
    string result = breadthfirstsearch(board, target);
    char finalBoard[4][3];
    fromstring(result, finalBoard);
    cout << "Target state:" << endl;
    printboard(finalBoard);
    return 0;
}
```

Complexity

Time complexity: $O(V \log V)$

Space complexity: $O(V)$

V: Number of different states

Code implementation (Simulated Annealing):

```
void simulatedAnnealing(char board[4][3], string target) {
    vector<string> path;
    int restarts = 0;
    int MAX_RESTARTS = 200;
    double temperature = 1000.0;
    double coolingRate = 0.995;
```

```

double minTemperature = 0.1;

string currentState = fromboard(board);
path.push_back(currentState);

while (true) {
    if (currentState == target) {
        cout << "Solution found in " << path.size() << " steps." << endl;
        cout << "Final state:" << endl;
        printboard(board);
        return;
    }

    vector<string> nextStates = nextstate(board);
    if (nextStates.empty()) continue;

    string randomState = nextStates[rand() % nextStates.size()];
    char randomBoard[4][3];
    fromstring(randomState, randomBoard);

    int currentScore = score(board, target);
    int randomScore = score(randomBoard, target);
    int delta = randomScore - currentScore;

    if (delta > 0 || (double)rand() / RAND_MAX < exp((double)delta / temperature)) { //a luck
        chnace to see if to accept the worst state
    }
}

```

```

        currentState = randomState;

        fromstring(currentState, board);

        path.push_back(currentState);
    }

    temperature *= coolingRate; // decreasing the tolerance as time goes

    if (temperature < minTemperature) { // if temprature is too low, we are likely stuck in a
local minimum, so we restart

        restarts++;

        if (restarts > MAX_RESTARTS) {

            cout << "No solution found." << endl;

            return;

        }

        temperature = 1000.0; // reset the tempreture to strat from a new place

        for (int k = 0; k < 40; k++) {

            currentState = randomshuffle(currentState);

        }

        fromstring(currentState, board);

        path.clear();

        path.push_back(currentState);

    }

}

}

```

```

int main() {
    srand(time(nullptr));

    char board[4][3] = {
        {'B','B','B'},
        {'.','.', '.'},
        {'.','.', '.'},
        {'W','W','W'}
    };

    cout << "-----Simulated Annealing-----" << endl;
    cout << "Starting state:" << endl;
    printboard(board);
    cout << "-----" << endl;
    string target = "WWW.....BBB";
    simulatedAnnealing(board, target);
    return 0;}

```

Complexity

Time complexity: $O(R \times I)$

Space complexity : $O(1)$ // doesn't save past movements it overwrites the current one it had

R = max restarts

I = Iterations per restart

Methodology:

Hill climbing:

- It takes the input state of the game
- It then goes to next step function and takes the available steps for this state
- And takes the best state from them and keeps on doing so until it doesn't find a better scoring or equal state
- It tries to move sideways to see better options
- clears the path when it restarts
-
- It then randomizes the current state to try and escape the local minimum it reached and do the steps from first again it has R number of retries that if reached it stops meaning it can't find the goal

Simulated Annealing:

- It takes the input state of the game
- It then goes to next step function and takes the available steps for this state
- It sees if the new step is better if it is it takes it and decreases the tolerance(temperature) so it becomes stricted
- If not it tries an equation of $P = e^{-(\Delta E / T)}$ { Where:
P = probability of accepting the worse state
 ΔE = delta = newScore – currentScore (negative value since worse state)
T = current temperature}
- If the random number it pulled is smaller than the output of this equation it takes the worst state hoping It would help getting the goal
- Then it decreases the tolerance(if better is found or worse)
- When tolerance reaches 0.1 it stops

BFS:

- It takes the input state of the game
- It adds the initial state to a queue and marks it as visited so it doesn't visit the same state twice and fall into an infinite loop
- It then takes the current state from the front of the queue and goes to the next state function to take all available steps for this state
- For each new state if it was not visited before it adds it to the queue and marks it as visited and records its parent so we can track the path later
- It keeps on doing so level by level meaning it tries all possible moves of the current state before going deeper
- This level by level approach is what guarantees it always finds the shortest path to the goal
- It keeps trying every single possibility until it finds the goal state

Comparative Evaluation:

Algorithm **BFS Hill Climbing Simulated Annealing**

Completeness   

Optimality   

Solves Puzzle   

When trying the iterative improvement algorithm and using the hill climbing algorithm it didn't reach the target state because it was always looking for the next best move from its current position which wasn't optimal because in this problem it need to sacrifice a better solution so we can reach the goal or to make space for the next piece and that wasn't possible because the scoring system rewarded it for being closer to its goal without thinking about the other pieces and even when I tried shuffling the puzzle so it maybe finds a different minimum because it may have been stuck on a local minimum and couldn't find the global minimum it also didn't help even with 100000 tries and making it accept equal states not better and making it try sideways having 50 times trying sideways it wasn't reaching the goal and that was down to the algorithm way of functioning which is looking at the point you are at and looking for the next best one from current position so that wasn't working for this puzzle. While in BFS it didn't care for the best movement it saw all possible states the game gets from every play and then it found the solution. Although Hill climbing has better space complexity but it doesn't reach the optimal solution like BFS and Simulated Annealing. On the other hand Simulated Annealing is an iterative improvement algorithm that tolerates worse states by using an equation that punishes very bad states or too much bad states and its tolerance decreases as time passes but it reaches the solution because it doesn't always take the best sometimes it settles for bad ones to reach the solution

Alternative Approaches:

While BFS successfully solved the knight swap puzzle and Hill Climbing failed, there are other algorithms that could have been used to address this problem:

1. A* Search: A* is the most suitable alternative for this puzzle. It combines the completeness and optimality of BFS with a heuristic function to guide the search, making it faster and more memory efficient than BFS. It evaluates each state using the formula $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach the current state and $h(n)$ is the estimated cost to reach the goal. This would make it significantly faster than BFS while still guaranteeing a solution.

2. Depth First Search (DFS): DFS explores one path fully before backtracking to try another. It uses much less memory than BFS since it only stores the current path, however it does not guarantee the shortest solution and may get lost in very long paths before finding the goal.

Most Efficient Alternative: Among both approaches, A Search* would be the most efficient algorithm for this puzzle as it guarantees finding the optimal solution like BFS while being faster and using less memory due to its heuristic guidance.

Hill climbing output:

```
starting state:  
B B B  
. . .  
. . .  
W W W  
Maximum restarts reached. NO solution found.  
Total restarts attempted: 201  
tried 7 steps.
```

```
starting state:  
B B B  
. . .  
. . .  
W W W  
Maximum restarts reached. NO solution found.  
Total restarts attempted: 501  
tried 5 steps.
```

```
starting state:  
B B B  
. . .  
. . .  
W W W  
Maximum restarts reached. NO solution found.  
Total restarts attempted: 1001  
tried 5 steps.
```

Here the input is the starting state board as shown in the image and no matter how many times we increase the maximum number of restarts it keep on not reaching the solution due to it not sacrificing the best option to reach the optimal solution

BFS output:

```
-----Breadth-First Search-----  
starting state:  
B B B  
. . .  
. . .  
W W W  
-----  
Target state:  
W W W  
. . .  
. . .  
B B B
```

Here we input the starting state of the board and it reached the target state always because it looks for all solutions of this board

Simulated Annealing output:

```
-----Simulated Annealing-----  
Starting state:  
B B B  
. . .  
. . .  
W W W  
-----  
Solution found in 603 steps.  
Final state:  
W W W  
. . .  
. . .  
B B B
```

```
-----Simulated Annealing-----  
Starting state:  
B B B  
. . .  
. . .  
W W W  
-----  
Solution found in 564 steps.  
Final state:  
W W W  
. . .  
. . .  
B B B
```

```
-----Simulated Annealing-----  
Starting state:  
B B B  
. . .  
. . .  
W W W  
-----  
Solution found in 519 steps.  
Final state:  
W W W  
. . .  
. . .  
B B B
```

In Simulated Annealing it takes the initial state as input it always reaches the target but not always in the same steps that's due to the randomizing that it can take a state that makes it reach faster sometimes and sometimes not.

Conclusion:

In this task, the knight swap puzzle was addressed using multiple algorithms to evaluate their effectiveness. The required iterative improvement algorithm, Hill Climbing, failed to reach the target state in every attempt. This was due to the nature of the puzzle which requires temporarily sacrificing better states to make space for other pieces, causing Hill Climbing to get permanently stuck in local minimum rather than reaching the global minimum.

Simulated Annealing proved to be a significantly better iterative improvement alternative, as it successfully solved the puzzle by accepting worse states with a decreasing tolerance, allowing it to escape local minimum effectively. This makes it the most suitable iterative improvement algorithm for this problem.

When compared to search-based algorithms, BFS guaranteed finding a solution but had higher memory usage. Among all algorithms discussed, A* Search would theoretically be the most efficient overall solution as it combines the completeness and optimality of BFS with a heuristic to reduce memory usage and search time.

In conclusion, not all iterative improvement algorithms are suitable for every problem. The choice of algorithm should be based on the nature of the problem, and in this case Simulated Annealing was the best iterative improvement approach while A* would be the best overall approach.

References:

For Simulated Annealing: <https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/>

For Hill climbing: <https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/>